

condaEnvironment

September 10, 2026

1 Conda Environments

An environment is workspace where you can set which software packages/libraries (e.g., Numpy, Matplotlib, Scipy, etc.) and even the python version.

Fun analogy from Gemini: *“Think of it like a specialized kitchen. If you want to cook Italian food, you populate your kitchen with basil and pasta. If you want to switch to baking pastries, you don’t want the garlic and fish sauce from yesterday ruining your dough—you’d rather step into a clean, separate baking station.”*

Note: conda is an environment (and package) manager mamba might be a faster environment manager... I think most conda commands can be done in mamba by replacing “conda” with “mamba” let me know if there are issues

1.1 Why should I create an environment?

- Preserve packages when you run a project, and differentiate them between projects. Some packages between projects can conflict.
- Makes your project more replicable. Reviewers can ask for your source code, they’d want to also know what requirements are needed to run it.
- Lessen the need of making changes due to updates in packages

1.2 Note: anything in the [] should be changed to a user-defined name (including the square brackets)

1.2.1 Ex. if the command is `conda create -n [insertEnvironmentNameHere]`, then type `conda create -n AMeaningfulEnvironmentName`

1.3 Creating conda environments

Note: make sure that in the terminal, you’re in the `home/jovyan` folder? (Note sure if this is necessary)

1. `conda init bash` -> to initialize the shell to activate and deactivate conda environments later
2. `conda create -n [insertEnvironmentNameHere]` -> creates a blank environment
 1. Alternative: `conda create -n testCondaEnv --clone [pathOfTheEnvironment]` -> clones an already made environment
 2. Alternative: `conda env create -f [pathOfFileWithEnvironment.yml]` -> creates an environment based on a file that has a list of packages to install (preferably a `.yml` file)

3. when running the conda create command, press y then enter if there's a prompt like do you want to install in this folder or something (I forgot the exact command; to edit later)... You can press enter and it will use the default [y] choice, but I want to avoid wreckless enter-pressing.
4. `conda rename -n [old_name] [new_name]` -> if you want to rename your environments

1.4 Activating and Deactivating

1. `conda env list` -> list the environments present in your shell
2. `source activate [environmentName]` -> activates your enviroment (supposedly its `conda activate [envName]`, but `source activate ...` works for me)
3. `conda deactivate/ source deactivate` -> deactivates your current environment
4. `conda list` -> list all the packages installed in your current environment

1.5 Installing packages

1. `pip install [packagename]=[version]/ conda install [packagename]=[version] / mamba install [packagename]=[version]` -> depends on the package you want to install, check its documentation (usually you can copy the command)
2. Examples: (practice creating a blank environment, and installing the following packages on them)
 1. Installing Numpy: <https://numpy.org/install/> (click the environment-based tab)
 2. Installing Matplotlib <https://matplotlib.org/stable/install/index.html> (can use conda option)

1.6 Making the package appear as an option in the jupyter notebook kernel selection

1. `pip install ipykernel/conda install ipykernel` -> install ipykernel
2. `python -m ipykernel install --user --name [yourCurrentEnvironmentName] --display-name "Python [NameToAppear]"` -> make it appear in jupyter notebook as "Python [NameToAppear]"

1.7 Creating a file that lists packages in a file

- `conda env export > [fileName].yaml` -> you can use this file to create an enviroment with the `conda env create -f [pathOfFileWithEnvironment.yaml]` option

[]: